Agentic AI System for Sikka API Integration — Payment System Use Case

Overview

This report outlines the detailed approach and implementation of an Agentic AI system designed to assist developers in efficiently leveraging the Sikka API ecosystem. The primary focus is to build a Payment System use case, showcasing how the AI interprets user requirements, semantically matches them with relevant APIs, and generates usable application code. The solution utilizes Large Language Models (LLMs), Retrieval-Augmented Generation (RAG) techniques, and automated document processing pipelines for dynamic, scalable, and future-proof integrations.

Objectives

- Develop an AI agent capable of understanding user instructions and mapping them to appropriate Sikka API endpoints.
- Advanced data processing and chunking are used to feed the Sikka API documentation into a vector store for semantic search.
- Design a modular, reusable pipeline that supports future extensions such as chat interfaces, backend generation, and wiring.

Tools and Technologies

- **Python**: Core language for system logic, data handling and AI agent logic.
- **Pandas**: Used to read and process the Excel sheet containing Sikka API metadata.
- **LangChain**: Provides the framework for document chunking and interaction with LLMs.
- **CharacterTextSplitter**: Used to split large raw text content into manageable overlapping chunks for improved semantic retrieval.
- **LLM (Ollama)**: To interpret user intent and generate corresponding code or endpoint usage.

Data Processing Pipeline

1. **Load API Metadata**
    - Input Source: Excel file containing API details (Name, Description, Endpoint URL, Documentation Link).
    - Method: Use *pandas.read_excel()* to load into a Data Frame.
2. **Format Rows**
    - Combine each row into a structured text snippet, ensuring clarity and consistency.

    API Name: GetPatientPayments | Description: Retrieves patient payment details | Endpoint: [URL] | Docs: [URL]
3. **Build Text Corpus**
    - Join all formatted rows into one large raw text string.
    - This corpus acts as the knowledge base for subsequent chunking and querying.
4. **Chunking for LLM Input**
    - Use *characterTextSplitter*
        - Chunk Size: 1000 characters.
        - Overlap: 200 characters.
    - Purpose: Maintain context continuity between chunks to enhance semantic retrieval accuracy.
5. **Resulting Output Snippet**
    - **Chunks ready to:**
        - Directly query using LLMs.
        - Feed into vector stores for embeddings-based search.

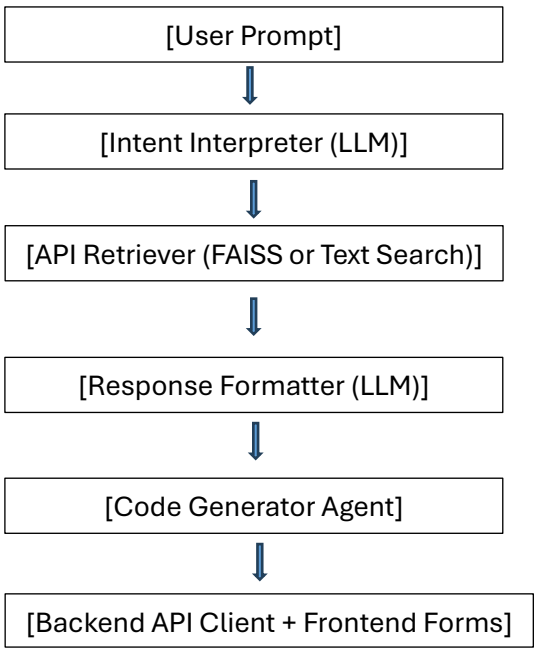
Example Output Snippet

API Name: GetPatientPayments | Description: Retrieves patient payment details | Endpoint: https://api.sikkasoft.com/v4/payments/details | Docs: https://apidocs.sikkasoft.com/#...

Design Considerations

- **Avoiding Hardcoded APIs**: The system does not rely on predefined API endpoints, making it generalizable to new or updated APIs.
- **Scalability**: Modular pipeline allows adding new APIs or document sources without altering the core logic.
- **Extensibility**: Can integrate with vector stores (e.g., FAISS) and RAG pipelines to improve query-based API discovery.
- **Low Latency Retrieval**: Plan to use FAISS to ensure millisecond retrieval times even with large API sets.
- **Fallback Handling:** In case no suitable API is found the system suggests close alternatives to the user.

Expanded Architecture Diagram



Detailed Agent Workflow for Payment System

- **Intent Parsing:** LLM analysis user instruction: “Build a payment system”.
- **Semantic Search:** Find APIs mentioning “payment”, “charge”, “transaction”.
- **API Recommendation:** Suggest an endpoint like *GET /payments/details*, *POST /payments/add*
- **Backend Code Generation:** Create server-side functions (Python, Node.js) to interact with APIs.
- **Frontend Code Generation:** Generate UI components: Payment Form, Payment History Table.
- **Critique and Iteration:** Optional second LLM agent reviews and optimizes the code.

Next Steps and Future Enhancements

- **Integrate with Vector Store**: Embed API documentation for faster and more relevant search.
- **Conversational Agent Layer**: Use multi-turn conversations with agents like CrewAI or AutoGen.
- **Add Testing Pipelines**: Auto-generate API integration test scripts based on an endpoint.
- **Frontend UI for Developers**: Build a simple web app using Streamlit, Next.js, or Flask for users to query the agent.
- **Multi-model Inputs:** Accept screenshots, API docs, and user queries in mixed formats.
- **Self-healing APIs:** If an API is deprecated or unavailable, suggest fallback or alternatives automatically.

Conclusion

This system provides a robust foundation for building an intelligent assistant that can streamline Sikka API adoption.

By focusing on automation, scalability, and extensibility, the architecture ensures developers spend more time innovating and less time deciphering complex documentation.

Through this Payment System case study, we demonstrate that an Agentic AI approach, empowered with modular pipelines and RAG techniques, can substantially accelerate full-stack development for real-world applications.